LEARNEROS

Backend Architecture & Operations

Routing, APIs, persistence, assessment context, insight lifecycle, LLM orchestration, caching, and deployment

SYSTEM	LearnerOS adaptive learning backend
REPOSITORY	/Users/srichandrasamanapalli/code/AI TUTOR/learneros/backend
REFERENCE DATE	2026-08-10
STATUS	Code-grounded implementation reference; assessment behavior updated after the subsection/concept-scope changes

Reading guide
This document describes the backend currently represented by the source tree. Assessment behavior is grounded in the production router and its focused regression tests. Unrelated evaluation, export, promotion, and observability scripts are outside the scope of this revision.

Contents

1. System Role And Architecture

LearnerOS is an adaptive learning backend. It combines a curriculum knowledge graph with a student-specific evidence layer. The curriculum graph describes grades, subjects, textbooks, chapters, sections, subsections, concepts, prerequisite edges, and related learning paths. The evidence layer records what a particular student appears to understand, misunderstand, or partially understand after assessments and tutoring interactions.

The backend is a FastAPI service backed by Neo4j. LLM calls use OpenAI-compatible clients for Fireworks, Cerebras, and Gemini. Firebase Authentication supplies identity and role claims. OpenTelemetry is optional and exports traces and metrics through OTLP when enabled. The service is deployed as a single Uvicorn process on Fly.io with a health check on port 8000.

- **Curriculum graph:** stable, non-personalized learning structure used for textbook navigation, prerequisite-aware teaching, graph views, and retrieval.
- **Student evidence graph:** student-scoped insights, supersession chains, embeddings, assessment attempts, and tutor conversations.
- **Generation layer:** written-question and MCQ generation are target-subsection scoped; evaluation uses the target subsection plus full-section reference context and then creates validated, concept-linked insights.
- **Analytics layer:** risk scoring, institute-scoped teacher dashboards, active insight summaries, and semantic student clusters.

Runtime topology

```
FastAPI -> routers -> services -> Neo4j / LLM providers / OTLP
                |
                +-> R2/public asset redirects for animations and chapter covers
```

2. Runtime Request Lifecycle

Every request enters through the FastAPI application in backend/app/main.py. Middleware establishes or propagates an X-Request-ID, emits structured request-start and request-complete events, and records total latency. Unhandled exceptions are logged with a bounded message and traceback and are converted into a generic 500 response by FastAPI.

- Request ID: accepts a safe caller-provided value matching [A-Za-z0-9_-]{8,128}; otherwise generates a UUID-derived ID.
- CORS: configured from FRONTEND_URL and CORS_ORIGINS, with localhost development origins included. Authorization, content type, accept, and origin headers are allowed; X-Request-ID is exposed.
- Telemetry: configure_telemetry(app) runs at import time. If OTEL_ENABLED is false, the instrumentation path is a no-op.
- Database: the Neo4j driver is lazy and shared as a singleton. Startup does not require a successful database query, so a later request can be the first connection attempt.
- Shutdown: closes the Neo4j driver and flushes OpenTelemetry providers.

STAGE	WHAT HAPPENS	FAILURE BEHAVIOR
HTTP entry	CORS, request ID, structured start event	CORS rejection occurs before application logic; request ID remains available for logs
Auth dependency	Firebase bearer token verification and role/institute checks	401 for missing/invalid token; 403 for insufficient role or missing institute claim
Context load	Neo4j read of section/subsection content plus direct-section or chapter-fallback concept candidates	Missing section content returns 404; a subsection outside the section returns 400; zero valid concept candidates is allowed
LLM stage	OpenAI-compatible provider request and schema parsing	Configured retries; provider fallback only where the operation supplies fallback targets
Persistence	Neo4j writes for attempts, insights, conversations, embeddings, and clusters	Assessment persistence is designed to be non-fatal to learner-facing evaluation; failures are recorded in telemetry
Response	JSON payload plus cache and request headers where applicable	Completion event includes status and latency

3. Routing And API Surface

The route registration source of truth is backend/app/main.py. Router modules group endpoints by domain. Most Neo4j identifiers use the :path converter. The section-concepts route deliberately uses /sections/{section_id}/concepts without :path so it cannot absorb the /sections/{id}/test/... namespace; section IDs contain colons but no slash.

DOMAIN	REPRESENTATIVE ROUTES	PURPOSE
Health	GET /health	Fly health check and basic liveness
Auth	GET /auth/me; DELETE /auth/me	Current Firebase identity and profile deletion

DOMAIN	REPRESENTATIVE ROUTES	PURPOSE
Curriculum	GET /api/grades; /grades/{grade}/subjects; /chapters/{id}/graph	Public textbook structure and graph reads
Lineage	GET /api/concepts/{concept_id}/lineage	Concept -> grade -> subject -> chapter lineage, capped for performance
Student insights	GET /api/students/me/insights; /search; /concept/{id}	Active student insight retrieval
Insight actions	POST /api/students/me/insights/{id}/explain; /test/mcq; /test/mcq/evaluate	Remediation and insight-specific testing
Assessment	GET /api/sections/{id}/test/question; /test/mcq; POST .../evaluate	Target-subsection question/MCQ generation; personalized evaluation with validated insight scope
Exercises	POST /api/sections/{id}/test/exercises/evaluate	Image or text exercise evaluation and assessment persistence
Section tutor	POST /api/sections/{id}/tutor/chat	Subsection/section-context tutor
Global tutor	POST /api/tutor/chat; GET /api/tutor/sessions	Textbook-agnostic tutor with active insight retrieval
Institutes	GET /api/institutes; PATCH /api/students/me/institute	Institute association and profile context
Teacher applications	POST /api/teachers/apply; admin review routes	Teacher access request and approval workflow
Teacher dashboard	GET /api/teachers/me/dashboard/{overview,students,clusters}	Institute-scoped analytics and cluster reads
Admin	POST /api/admin/institutes; users/claims; rebuild-clusters	Institute, Firebase claim, and cluster administration

Public and personalized boundaries

- **Public curriculum reads:** grades, subjects, chapters, sections, graph structure, concept lineage, exercises, and non-personalized content are readable without student identity.
- **Student-scoped reads/writes:** student insights, tutor sessions, test evaluation, exercise evaluation, institute/profile changes, and assessment attempts require the current student identity.
- **Teacher-scoped reads:** teacher dashboard routes require role=teacher and an institute_id Firebase claim; every analytics query is constrained to the claim institute.
- **Admin-scoped writes:** institute administration, role claims, teacher application decisions, and cluster rebuild triggers require role=admin or role=superadmin.

4. Authentication, Authorization, And Tenant Isolation

backend/app/auth.py is the authentication boundary. Firebase Admin verifies bearer ID tokens. The token claims, not a user-provided request field, determine identity and role. Domain dependencies then map the verified token to a student, teacher, or admin context.

DEPENDENCY	REQUIRED CLAIMS	SIDE EFFECTS / SCOPE
get_current_user	Valid token; role missing, empty, or student	Ensures Student node with id student:{uid}; returns student identity
get_authenticated_user	Valid token	Role-neutral identity for flows that must not create student side effects
get_optional_user	No token allowed; valid student token optional	Anonymous is supported; elevated role is not silently treated as a student

DEPENDENCY	REQUIRED CLAIMS	SIDE EFFECTS / SCOPE
get_current_teacher	role=teacher and institute_id	Ensures Teacher audit node; returns teacher_id and institute_id
get_current_admin	role=admin or superadmin	Returns admin identity for administrative actions

- Student identity is normalized as student:{firebase_uid}; teacher identity as teacher:{firebase_uid}.
- Teacher analytics never trusts institute_id from query parameters or request bodies; it uses the verified claim.
- Student detail routes verify that the target Student node belongs to the teacher's institute before returning data.
- The backend does not treat an admin claim as a teacher claim. A dual-role account needs an explicit role policy; the current teacher dependency expects role=teacher.

Operational requirement

Firebase custom claims must be refreshed in the client after an admin changes a role. The old ID token continues to carry the old claims until refresh or reauthentication.

5. Neo4j Data Model And Persistence

Neo4j is the system of record for curriculum structure and learner state. backend/app/database.py creates one shared driver with 30-second connection and acquisition timeouts, a 30-second transaction retry window, one-hour connection lifetime, keep-alive, and a pool size of 50.

Core curriculum nodes and relationships

- Grade -> Subject -> Textbook -> Chapter -> Section -> Subsection is the primary navigation hierarchy.
- Sections connect to Concept nodes through REQUIRES. Assessment first uses concepts directly required by the active section; if none exist, it collects concepts required by sections in the same chapter as evaluation-only fallback candidates.
- Exercises belong to sections and may be associated with concepts where the curriculum data provides that linkage.

Learner-state nodes and relationships

ENTITY	KEY FIELDS	RELATIONSHIPS / USE
Student	id, uid, name, email, role, grade, institute_id	ENROLLED_IN Institute; SUBMITTED AssessmentAttempt; owns Insight and tutor sessions
Insight	id, type, category, content, is_active, embedding, created_at	ABOUT_CONCEPT, ABOUT_SOURCE, ABOUT_SUBSECTION; SUPERSEDES prior insight
AssessmentAttempt	id, mode, answer metadata, status, scores, timestamps	SUBMITTED by Student; ABOUT_SECTION/SUBSECTION/EXERCISE; ASSESSED Concept
TutorConversation / Message	session, role, content, timestamps	Student-owned conversation history; loaded into LangGraph state
StudentClusterSnapshot	id, institute_id, run_at, algorithm, quality_json	Contains StudentCluster nodes; historical snapshots retained
StudentCluster	id, label, size, risk metadata, explanations	IN_SNAPSHOT; Student -[:IN_CLUSTER]-> cluster with confidence/distance

Persistence patterns

- Read queries use session.execute_read; write queries use session.execute_write, allowing the Neo4j driver to apply transaction retries.
- Insight writes create a new evidence record and connect it to the relevant source/concept/subsection. Reconciliation then changes active state and creates lineage metadata rather than deleting history.

- Assessment attempts are privacy-conscious: image answers persist count and SHA-256 hashes rather than raw image bytes; text is not stored for image-mode answers.
- Assessment persistence is queued/attempted outside the critical learner response path. A persistence failure is observable and should not make a successful model evaluation look like a model failure.

6. Insight Lifecycle And Reconciliation

Insights are student-specific evidence attached to curriculum concepts and sources. The active state is a query projection over historical evidence, not a replacement for historical records. The written-answer and MCQ flows are: resolve target subsection -> evaluate against subsection content -> select valid concept candidates -> validate structured evidence -> insight creation -> reconciliation -> embedding -> persistence.

Insight pipeline

```
Student answer
  -> resolve and validate the requested subsection inside the section
  -> evaluate against target subsection content; full section is reference context only
  -> use direct section concepts, or chapter concepts when the section has none
  -> reject insight IDs outside the resolved candidate set
  -> attach accepted evidence to the target subsection/source and concept
  -> reconcile against active insight(s) for the same student + source/concept key
  -> preserve history, embed the active insight, and retrieve it later for teaching/tutor context
```

Evidence categories

TYPE	MEANING	TYPICAL DOWNSTREAM EFFECT
MISCONCEPTION	Evidence of an incorrect or conflicting mental model	Prioritize contrastive explanation, diagnostic testing, and risk score +5
PARTIAL_UNDERSTANDING	Correct fragments with an unresolved conceptual gap	Targeted example/prerequisite review and risk score +2
COMPETENCY	Evidence of reliable understanding or application	Reduce risk score by 2 and allow advancement/deepening
TAUGHT	Supported type for instructional evidence	Type exists in the model; active runtime routes do not currently create it

Reconciliation semantics

- **Merge:** when new evidence is about the same underlying concept/source, the system can preserve the older true evidence and produce a coherent updated active record rather than blindly replacing it.
- **Replace/supersede:** when the new evidence materially updates or contradicts the prior statement, the new insight becomes active and the previous record remains queryable through its lineage.
- **Active state:** active insight reads filter is_active=true and exclude superseded records from the current summary while historical chains remain available for teacher timelines and audit.
- **Embedding:** the new insight content is embedded with Fireworks qwen3-embedding-8b when the embeddings key is available; embeddings support semantic matching for tutor and clustering paths.

Design implication

A newer insight does not logically make every older observation false. Reconciliation must preserve compatible evidence and only supersede what the new statement actually updates. The current graph model preserves lineage, but the quality of the final summary still depends on the model's merge/replace decision.

7. LLM Orchestration And Prompt Paths

backend/app/services/llm.py provides one provider-neutral LLMService. It uses the OpenAI Python SDK against OpenAI-compatible endpoints and caches clients by provider and purpose. Provider selection is explicit for most production paths and can be configured through environment variables.

PROVIDER	PURPOSE	CONFIGURATION
Fireworks	Default generation, embeddings, production model calls	FIREWORKS_API_KEY; FIREWORKS_BASE_URL; FIREWORKS_MODEL; FIREWORKS_API_KEY_EMBEDDINGS; FIREWORKS_EMBEDDING_MODEL
Cerebras	Optional fast generation target/fallback	CEREBRAS_API_KEY; CEREBRAS_BASE_URL; CEREBRAS_MODEL
Gemini	Exercise image/text evaluation path	GEMINI_API_KEY; GEMINI_OPENAI_BASE_URL; EXERCISE_EVAL_MODEL=gemini-3.1-pro-preview

Production LLM operations

- **generate_text:** accepts system/developer/user messages, optional image data URLs, optional JSON mode, model/provider selection, and operation labels.
- **generate_json:** calls the model, parses JSON, validates the expected shape in the caller, and retries parse/generation failures.
- **embed:** creates an embedding for insight or query text through Fireworks; the embedding operation is separated from generation purpose.
- **Provider fallback:** is not universal. It is used only when the caller supplies fallback targets; a provider misconfiguration can therefore fail the operation rather than silently switching models.
- **Content capture:** prompt/response/image content is fingerprinted or hidden in telemetry by default; full content capture requires OTEL_CAPTURE_CONTENT=true.

Context by operation

OPERATION	CONTEXT SENT	PERSONALIZATION
Section question/MCQ generation	Target subsection content as primary scope; complete parent section as definitions/surrounding reference only; no concept candidates in the generation prompt	Non-personalized and share-cache eligible; requested subsection ID is authoritative
Section/test evaluation	Question/answer, target subsection ground truth, full-section reference context, and direct-section or chapter-fallback concept candidates	Personalized evaluation; accepted insights must use an exact candidate concept ID and the target subsection source ID
Exercise evaluation	Exercise question plus text or image input and chapter prerequisite concept context	Personalized; Gemini 3.1 Pro Preview path
Insight explain/test	Insight text plus source/concept context	Personalized remediation; not cached
Section tutor	Subsection/section content plus top matched active insights and conversation state	Personalized; LangGraph-backed
Global tutor	User query plus top matched active insights and persisted conversation state	Textbook-agnostic; personalized only when active insight retrieval matches

8. Retry, Timeout, And Failure Semantics

The backend has several retry boundaries, each with a different purpose. Retrying a provider call is not the same as retrying a Neo4j transaction, and neither should be confused with a client retry. The current LLM retry loop is immediate; it does not sleep between attempts.

BOUNDARY	CURRENT BEHAVIOR	IMPORTANT CONSEQUENCE
LLM JSON generation	Caller/service retries usually default to 2 retries after the initial attempt	Can multiply provider latency; retries are immediate and should be bounded by endpoint timeout
Embedding	Embedding helper retries usually default to 2 retries	Embedding failure should not erase a valid insight; downstream retrieval may have less signal
Gemini exercise evaluation	Gemini JSON helper uses approximately 90-second timeout and up to 2 retries	Image evaluation can be slow; keep outside cache and monitor stage latency
Neo4j transaction	Driver max_transaction_retry_time is 30 seconds	Transient connection/transaction failures may be retried by the driver
Remote cache	Upstash errors fail open to local cache/generation	Cache outage should not become a learner-facing 500
Assessment persistence	Persistence failures are captured separately and do not necessarily fail evaluation response	Telemetry must distinguish model success from persistence success

Failure classification

- Validation failure: model returned malformed JSON or invalid enum/category; retry while attempts remain, then return a controlled error.
- Provider failure: timeout, rate limit, unavailable model, or invalid credentials; record provider/model and operation in structured telemetry.
- Persistence failure: Neo4j write failed after evaluation; keep an explicit persistence_failed/dead_letter_recorded assessment state where available.
- Authorization failure: do not retry; the token, role, or institute scope is invalid.
- Cache failure: do not fail the request solely because local/Upstash cache is unavailable; regenerate or use the origin path.

Recommended operational improvement

Add bounded exponential backoff with jitter to provider retries, preserve the current attempt count in telemetry, and keep the total retry budget below the HTTP request timeout. Immediate retries can amplify rate limits during provider degradation.

9. Caching And Content Delivery

Caching is intentionally split by personalization. Stable curriculum and non-personalized generation can be shared; student insights, evaluations, and tutor responses must not be shared across users.

LAYER	CURRENT SCOPE	TTL / BEHAVIOR
Curriculum in-process	Grades, subjects, chapters, sections, subsections, exercises, concepts, graphs, lineage	In-memory TTL/cache helper; public read headers are set by curriculum routes
Generation local cache	GET section question and GET section MCQ	Thread-safe LRU TTL; default 7 days; max 5,000 entries
Generation remote cache	Same non-personalized generation keys when Upstash is configured	Upstash REST /get and /set with EX; remote failures fail open
Frontend/session cache	Generation GET requests only	Short session-level cache and in-flight request de-duplication
Cloudflare edge	Public cacheable curriculum/generation responses when rules and headers permit	Depends on Cloudflare cache rules, cache-control, and route configuration
R2 assets	Chapter covers and animation HTML assets	Public deterministic URLs; not learner state

Generation cache key

The deterministic question/MCQ generation key includes endpoint type/version, section ID, requested subsection ID, variant, provider/model/fallback targets, and prompt schema version. It deliberately excludes auth identity and concept ID because concepts do not drive question generation. The v2 endpoint namespace prevents older, broader prompt results from being reused.

Generation response headers

```
Cache-Control: public, max-age=300, s-maxage=604 800, stale-while-revalidate=86 400
CDN-Cache-Control: public, max-age=604 800, stale-while-revalidate=86 400
X-Gen-Cache: HIT | MISS
X-Gen-Cache-Key: <short hash>
```

Never cache these responses publicly

Student insight lists, insight explain/test/evaluate responses, section/global tutor chat, assessment evaluation, exercise evaluation, profile/institute responses, teacher analytics, and admin operations are personalized or sensitive. They should use private/no-store or authenticated response behavior.

10. AI Tutor And LangGraph State

The tutor router exposes two related experiences. The section tutor receives textbook context for the current section/subsection. The global tutor is textbook-agnostic and uses semantic retrieval to bring in active student insights only when the query is close enough. Both persist conversation history in Neo4j.

Tutor flow

```
User query
  -> create/load conversation session
  -> embed query
  -> retrieve top active student insights
  -> assemble section or global context
  -> LangGraph state node invokes LLM
  -> persist user message + assistant message
  -> return response and matched insight/concept metadata
```

- LangGraph is used for stateful tutor interaction, not for curriculum extraction or teacher clustering.
- Conversation state is keyed to the authenticated student and session; latest-session and session-list endpoints read persisted history.
- Matched insights are active-only retrieval context. Superseded evidence remains history but is not injected as current learner state unless a specific historical route asks for it.
- The frontend renders Markdown-style content; model output is not instructed to avoid Markdown. Tables and math need a renderer that understands the returned syntax.
- Tutor response generation is not in the shared generation cache because it is user-specific and conversational.

11. Teacher Analytics And Semantic Clustering

Teacher endpoints are institute-scoped. The dashboard computes risk from active insights and reads persisted cluster snapshots. The clustering pipeline is manual/cron-driven through backend/scripts/build_teacher_clusters.py and can also be triggered through an admin endpoint.

Risk scoring

SIGNAL	WEIGHT	NOTES
MISCONCEPTION	+5	Active misconception evidence
PARTIAL_UNDERSTANDING	+2	Active partial evidence
COMPETENCY	−2	Offsets risk
Recency	1.25 / 1.0 / 0.75	<=7 days / 8-30 days / >30 days
Persistence	+2	Per active misconception with supersession depth >=2

Risk bands are HIGH for scores >=15, MEDIUM for 7-14, and LOW for <=6. Reason codes include HIGH_MISCONCEPTION_COUNT, PERSISTENT_MISCONCEPTIONS, and LOW_COMPETENCY_OFFSET.

Semantic clustering V2

- Only active insight embeddings are included.
- Student vectors are weighted means: misconception 1.0, partial understanding 0.6, competency 0.3, with recency decay.
- Students must pass minimum insight count and vector norm gates; vectors are L2-normalized before clustering.
- HDBSCAN is primary because it can find natural groups and noise. KMeans is an explicit fallback when the dataset is too small, HDBSCAN degenerates to one cluster/noise, or the algorithm fails.
- Noise is retained as an UNASSIGNED/NOISE cluster for teacher awareness rather than silently dropped.
- Snapshots preserve algorithm, engine version, eligible/noise counts, quality metrics, dominant concepts, terms, misconception statements, risk-band composition, and rule-based recommended actions.

SNAPSHOT FIELD	PURPOSE
algorithm / engine_version	Distinguishes hdbscan from kmeans_fallback and enables future rebuild comparisons
quality_json	Silhouette, noise ratio, size imbalance, and cluster entropy
eligible_students / noise_students	Explains coverage and sparse-data behavior
risk_band_counts_json	Shows intervention mix inside each cluster
top_concepts_json / top_misconceptions_json	Makes clusters teacher-actionable rather than embedding-only
IN_CLUSTER confidence/distance	Stores membership quality where the algorithm provides it

Why the dashboard can show no clusters

Snapshots are not created when an institute has no eligible students or when the cluster job has not run successfully. Cluster reads are snapshot reads; they do not build clusters synchronously during a dashboard request. The snapshot job must run against the same production Neo4j database used by the API.

12. Assessment Context, Concept Fallback, And Insight Scope

The production assessment contract separates question content from insight classification. Question and MCQ generation are controlled by the requested subsection content, with the full parent section supplied only as reference context. Concept candidates are introduced only during answer evaluation and insight creation.

STAGE	CONTENT BOUNDARY	CONCEPT BEHAVIOR
Metadata load	Collect ordered subsection IDs, titles, and content; build complete section reference text	Select concepts directly required by the section; if none, collect concepts required by sections in the same chapter
Target resolution	Require the submitted subsection ID to belong to the requested section	No concept is needed to resolve or render the subsection; invalid membership returns 400
Question / MCQ generation	Target subsection content is primary; full section may clarify notation, definitions, or surrounding context	Concept IDs and names are excluded from the generation prompt and cache identity
Answer evaluation	Judge only against the target subsection; do not require sibling-only facts	Expose either direct-section candidates or explicitly labeled chapter-fallback candidates

STAGE	CONTENT BOUNDARY	CONCEPT BEHAVIOR
Insight validation	Accept only evidence meaningfully supported by the tested question and learner answer	Reject concept IDs outside the resolved candidate set and invalid type/category values; an empty insight list is valid
Persistence	Force source_id to the validated target subsection	Persist only accepted concept links; unauthenticated or empty-insight cases are skipped without changing the evaluation result

Concept-scope selection

- Direct scope: if the active section has one or more direct REQUIRES Concept relationships, only those concepts are candidates for assessment insights.
- Chapter fallback: if the active section has no direct concepts, concepts required by other sections in the same chapter become evaluation-only candidates. This is the section 10.3 case.
- No candidates: if neither direct nor chapter concepts exist, the answer can still be evaluated, but returned insights are filtered to an empty list.
- Source integrity: every accepted insight receives the validated target subsection as source_id, regardless of what the model returns.

Focused regression coverage

Current contract checks in backend/tests/test_assessment_context.py

```
generation uses target content plus full-section reference and excludes concepts
subsection IDs outside the section are rejected before model evaluation
a section with no valid concepts evaluates successfully with empty insights
MCQ evaluation accepts an insight from the declared chapter-fallback scope
the curriculum concept route does not shadow the /test namespace
verified locally: 7 focused tests passed
```

Important boundary
The full section is sent because an individual subsection may omit definitions or nearby context. It is reference material, not permission to test sibling subsections. Likewise, chapter concepts are fallback labels for evaluation and insight creation; they do not broaden question generation.

13. Security And Privacy Controls

- Firebase ID tokens are verified server-side; the backend never trusts a client-supplied UID, role, or institute for protected operations.
- Teacher analytics always constrain Student and Insight queries by the teacher claim's institute_id.
- Public curriculum routes exclude student-specific fields. Personalized routes are authenticated and should not be edge-cached publicly.
- Raw exercise images are not persisted in AssessmentAttempt; only count and hashes are retained for audit/correlation.
- Tutor conversations are student-specific Neo4j records and are not included in public retrieval.
- R2 and provider credentials belong in deployment secrets, never in frontend bundles, committed source, or documentation.
- Safe request IDs and hashed content fingerprints reduce accidental leakage in logs.
- CORS is an origin control, not authentication. Production origins must be explicitly configured; wildcard origins with credentials would be unsafe.

Security review priorities

PRIORITY	ACTION
High	Keep production CORS allowlist narrow and verify the deployed API response includes the exact frontend origin for OPTIONS and authenticated requests.

PRIORITY	ACTION
High	Rotate any provider/R2 token that has ever been pasted into chat, screenshots, shell history, or committed files.
High	Enforce authentication/authorization on every new personalized route before adding cache rules.
Medium	Add rate limits and request-size limits consistently to expensive LLM and image evaluation routes.
Medium	Add explicit schema validation for all persisted insight categories and source/concept linkage.

14. Configuration And Deployment

Configuration is loaded by pydantic-settings in backend/app/config.py. The deployment image runs Uvicorn on 0.0.0.0:8000. Fly uses /health for service checks and currently has auto-stop disabled with one minimum machine in the repository configuration.

GROUP	VARIABLES	ROLE
Neo4j	NEO4J_URI, NEO4J_USER, NEO4J_PASSWORD	Graph database connection
Firebase	FIREBASE_SERVICE_ACCOUNT or FIREBASE_SERVICE_ACCOUNT_JSON	Server-side token verification and claims administration
LLM	GEMINI_API_KEY, FIREWORKS_API_KEY, FIREWORKS_API_KEY_EMBEDDINGS, CEREBRAS_API_KEY plus base URLs/models	Generation, evaluation, and embeddings
Telemetry	OTEL_ENABLED, OTEL_SERVICE_NAME, OTEL_EXPORTER_OTLP_*	OpenTelemetry traces/metrics
Generation cache	GEN_CACHE_TTL_SECONDS, GEN_CACHE_MAX_ENTRIES, GEN_PROMPT_VERSION	Local/remote generation cache policy
Upstash	UPSTASH_REDIS_REST_URL, UPSTASH_REDIS_REST_TOKEN	Optional shared generation cache
Frontend/CORS	FRONTEND_URL, CORS_ORIGINS	Allowed browser origins and deployment integration
R2 assets	ANIMATIONS_R2_PUBLIC_BASE_URL, CHAPTER_COVERS_ASSETS_DOMAIN, CHAPTER_COVERS_PREFIX	Public curriculum media URLs

Current Fly service shape

```
[http_service]
  internal_port = 8000
  force_https = true
  auto_stop_machines = 'off'
  auto_start_machines = true
  min_machines_running = 1

[[http_service.checks]]
  method = 'GET'
  path = '/health'
  interval = '30s'
  timeout = '15s'
```

Deployment dependencies

- Neo4j connectivity from Fly must be stable; a sleeping or cold remote database can create connection timeouts on the first request.
- The runtime image must listen on 0.0.0.0:8000, not localhost. Fly's health check is the first diagnostic for port/startup failures.
- Production CORS must include the exact deployed frontend origin(s), including custom domains and worker domains where both are used.

- R2 public domains are independent of backend deploys. Adding a new asset does not require a backend rebuild if the deterministic URL convention remains unchanged.
- OpenTelemetry export failures should not prevent application startup; the telemetry initializer is designed to fail soft.

15. Operational Runbook And Known Gaps

First checks when the app is slow or failing

- Check /health and Fly machine logs to distinguish process/port failure from a slow dependency.
- Inspect request latency and route in structured logs using X-Request-ID.
- Check Neo4j connection timeout/retry messages before changing frontend behavior.
- Check LLM provider/model configuration and retry counts for generation or evaluation failures.
- Check cache headers and X-Gen-Cache for non-personalized generation; do not add public cache rules to authenticated routes.
- For missing teacher clusters, inspect the cluster build report, eligible student count, algorithm status, and snapshot database identity.

Current implementation caveats

AREA	CURRENT CAVEAT	PRACTICAL EFFECT
LLM backoff	Retries are immediate, not exponential	Provider incidents can increase latency and rate-limit pressure
Generation cache	In-memory cache is per Fly machine; Upstash is optional	Multiple machines can have cold local caches; shared hit rate depends on Upstash configuration
Cluster snapshots	Built by manual/cron pipeline, not synchronously by dashboard	A working dashboard does not imply a snapshot exists
Chapter concept fallback	Fallback candidates can be broader than direct section links	Strict prompt and exact-ID validation are required; empty insights are a valid outcome
Teacher/admin roles	Dependencies use a single role value	A user with only admin role cannot automatically pass teacher dependency
Neo4j property warnings	Queries may reference optional properties not present on every node, such as logo	Warnings are usually non-fatal but schema cleanup is advisable
Global tutor	Insight retrieval is semantic and threshold/top-k dependent	No match is a valid result; the tutor should not invent personalization

Recommended next hardening steps

- Introduce a shared retry policy with exponential backoff, jitter, total deadline, and provider-specific retryability rules.
- Add explicit OpenTelemetry spans around Neo4j reads/writes and cache layers so dependency time is separated from route time.
- Add snapshot freshness and last-success metadata to the teacher API so the UI can distinguish empty data from stale pipeline data.
- Add a schema/relationship audit for direct section concepts and chapter-fallback coverage, including sections that intentionally have no concept links.
- Keep production-like route tests for both direct-concept and chapter-fallback sections, including the no-candidate case and cache-version changes.
- Keep public curriculum cache rules separate from authenticated API routes at the Cloudflare layer.

Source files
Primary implementation sources referenced by this document include backend/app/main.py, auth.py, config.py, database.py, routers/curriculum.py, routers/test.py, routers/tutor.py, services/llm.py, services/generation_cache.py, services/semantic_clustering.py, services/clustering_runner.py, teacher_analytics.py, and backend/tests/test_assessment_context.py. Unrelated eval/export/promotion/observability scripts are not used as behavioral evidence in this revision.